

Homework10 参考答案

Problem 1

要输出 beebabee..., 即每个 b 后面跟两个 e ($b:e = 1:2$)。用两个信号量 a、b, 让 t1 每打印一个 b 就放行两个 e, t2 每打印一个 e 归还一个 b 的许可, 而 t1 需要集齐两个许可才打印下一个 b。(P 即 sem_wait, V 即 sem_post。)

初始值: a = 0, b = 2。

```
void *t1(void *arg) {           // 打印 b
    while (1) {
        P(b); P(b);             // [1]
        printf("b");
        fflush(stdout);
        V(a); V(a);             // [2]
    }
}

void *t2(void *arg) {           // 打印 e
    while (1) {
        P(a);                   // [1]
        printf("e");
        fflush(stdout);
        V(b);                   // [2]
    }
}
```

正确性 (信号量值演变):

```
初始 a=0, b=2
t1: P(b)P(b) -> b=0, 打印 b, V(a)V(a) -> a=2
t2: P(a)      -> a=1, 打印 e, V(b)      -> b=1
t2: P(a)      -> a=0, 打印 e, V(b)      -> b=2
t1: P(b)P(b) -> b=0, 打印 b, V(a)V(a) -> a=2
...
```

输出 b e e b e e ... = beebabee...

Problem 2

每个线程拿到数组的一段 &a[M*i], 负责把这 M 个元素插入哈希表。doit 只需依次调用 Hash_Insert, 线程安全由 concurrent hash table 的“每个 bucket 一把锁”保证: 插到不同 bucket 可并行, 插到同一 bucket 串行。

```

void *doit(void *args) {
    int *base = (int *)args;
    for (int i = 0; i < M; i++) {
        Hash_Insert(&hashtb, base[i]);    // 用同一个全局哈希表 hashtb
    }
    return NULL;
}

```

配套的并发哈希表（每个桶一把锁）大致如下，使 List_Insert 线程安全：

```

typedef struct node { int key; struct node *next; } node_t;
typedef struct { node_t *head; pthread_mutex_t lock; } list_t;
typedef struct { list_t lists[BUCKET]; } hash_t;

void Hash_Init(hash_t *H, int buckets) {
    for (int i = 0; i < buckets; i++) {
        H->lists[i].head = NULL;
        pthread_mutex_init(&H->lists[i].lock, NULL);
    }
}

void List_Insert(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);    // 只锁这一个桶
    node_t *n = malloc(sizeof(node_t));
    n->key = key;
    n->next = L->head;
    L->head = n;
    pthread_mutex_unlock(&L->lock);
}

int Hash_Insert(hash_t *H, int key) {
    int bucket = key % BUCKET;
    return List_Insert(&H->lists[bucket], key), 0;
}

```

说明：题面 main 里写的是 Hash_Init(&hashtb_old, ...)，而线程插入的是全局 hashtb，这里统一成同一个表 hashtb（即对它 Hash_Init 后再让线程插入）。10 个线程各插一段，join 后全部插入完成；因为是按 bucket 加锁，不同 bucket 的插入真正并行，同一 bucket 的插入被串行化以避免 race。